

Reinforcement Learning under Threats

Victor Gallego

ICMAT-CSIC

victor.gallego@icmat.es

Roi Naveiro

ICMAT-CSIC

roi.naveiro@icmat.es

David Rios Insua

ICMAT-CSIC

david.rios@icmat.es

Abstract

In several reinforcement learning (RL) scenarios, mainly in security settings, there may be adversaries trying to interfere with the reward generating process. In this paper, we introduce Threatened Markov Decision Processes (TMDPs), which provide a framework to support a decision maker against a potential adversary in RL. Furthermore, we propose a level- k thinking scheme resulting in a new learning framework to deal with TMDPs. After introducing our framework and deriving theoretical results, relevant empirical evidence is given via extensive experiments, showing the benefits of accounting for adversaries while the agent learns.

1 Introduction

Markov decision processes (MDP) (?) provide a mathematical framework for modeling a single agent making decisions while interacting within an environment. We refer to this agent as the decision maker (DM, she). MDPs have been widely used to study reinforcement learning (RL) problems. More precisely, a MDP consists of a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, r)$ where $\mathcal{S}$ is the state space; $\mathcal{A}$ denotes the set of actions available to the agent; $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the transition distribution, where $\Delta(X)$ denotes the set of all distributions over set X ; and, finally, $r : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathbb{R})$ is the reward distribution (the utility the agent perceives from a given state and action). A common approach to solving MDPs is based on Q-learning, (?). In it, the agent maintains a table $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ that estimates the DM's expected cumulative reward, iterating according to the following update equation

$$Q(s, a) := (1 - \alpha)Q(s, a) + \alpha \left(r(s, a) + \gamma \max_{a'} Q(s', a') \right), \quad (1)$$

where α is a learning rate hyperparameter and s' is the state the agent arrives at after choosing action a in state s and receiving reward $r(s, a)$. While learning, the agent could choose actions according to a greedy policy ($\pi(s) = \arg \max_a Q(s, a)$) yet it is crucial to add stochasticity so that the agent can balance the exploration-exploitation trade-off, for instance with an ϵ -greedy policy.

However, when non stationary environments are considered, as when there are other learning agents that interfere

with the DM's rewards, Q-learning leads to suboptimal results (?).

1.1 Related Work

Several extensions of Q-learning in multi-agent settings have been developed in the literature, including minimax-Q (?), Nash-Q (?) or friend-or-foe-Q (?), to name but a few. We propose here to extend Q-learning from an Adversarial Risk Analysis (ARA), (?) perspective, in particular, through a level- k scheme (?), (?).

Within the bandit literature, the celebrated (?) introduced a non-stationary setting in which the reward process is controlled by an adversary. The adversarial machine learning literature has predominantly focused on the supervised setting (?). Other recent works tackle the problem of adversarial examples in RL (?; ?) though they focus on visual inputs.

Moreover, previous game-theoretical approaches to this problem have focused on modeling the whole multi-agent system as a game. Instead we shall face the problem of prescribing decisions to a single agent versus her opponents, augmenting the MDP to account for potential adversaries. We present such variant of MDPs, which we call Threatened MDPs (TMDPs), in the next section.

2 Threatened MDPs

In similar spirit to other reformulations of MDPs such as Constrained Markov Decision Processes (CMDP) (?) or Configurable Markov Decision Processes (Conf-MDP) (?), we propose an augmentation of the MDP to account for the presence of adversaries. In this paper, we restrict to the case of a DM facing a single opponent (he), leaving the extension to a setting with multiple adversaries for future work.

A *Threatened Markov Decision Process* (TMDP) is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{B}, \mathcal{T}, r, p_A)$ in which $\mathcal{S}$ is the state space; $\mathcal{A}$ denotes the set of actions available to the supported agent; $\mathcal{B}$ designates the set of threat actions, or actions available to the adversary; $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{B} \rightarrow \Delta(\mathcal{S})$ is the transition distribution; $r : \mathcal{S} \times \mathcal{A} \times \mathcal{B} \rightarrow \Delta(\mathbb{R})$ is the reward distribution (the utility the agent perceives from a given state and action pair); and $p_A(b|s)$ models the beliefs that the DM has about his opponent move, i.e., a distribution over $\mathcal{B}$ for each state $s \in \mathcal{S}$.

We propose to replace the standard Q-learning rule (Eq.1) by

$$Q(s, a, b) := (1 - \alpha)Q(s, a, b) + \alpha \left(r(s, a, b) + \gamma \max_{a'} \mathbb{E}_{p_A(b|s')} [Q(s', a', b)] \right) \quad (2)$$

and compute its expectation over the opponent's action argument

$$Q(s, a) := \mathbb{E}_{p_A(b|s)} [Q(s, a, b)]. \quad (3)$$

This may be used to compute an ϵ -greedy policy for the DM, i.e., choosing with probability $(1 - \epsilon)$ the action $a = \arg \max_a [Q(s, a)]$ or a uniformly random action with probability ϵ when the DM is at state s . Appendix ?? contains two lemmas showing that the previous update rules are fixed point iterations of contraction mappings.

However, in real life scenarios there will be uncertainty regarding the adversary's policy $p_A(b|s)$. Therefore, we propose using a level- k scheme (?) to learn the opponent model. In general, we consider both the DM and the adversary as rational agents that aim to maximize their respective expected cumulative rewards, though we start with a case in which the adversary is considered non-strategic (Section 2.1). Then, we go up a level in the level- k hierarchy, considering the adversary a level-1 agent and the DM a level-2 one (Section 2.2).

2.1 Non-strategic opponent

We begin by considering a stateless setting. The Q-function may be written then as $Q(a_i, b_j)$, with $a_i \in \mathcal{A}$ the action chosen by the DM, and $b_j \in \mathcal{B}$ the action chosen by the adversary. We assume that the supported DM is a joint action learner (i.e., she observes her opponent's actions after he has committed them). At every iteration, the DM shall choose her action maximizing her expected cumulative reward. However, she needs to predict the action b_j chosen by her opponent. A typical option is to model her adversary using fictitious play (FP), i.e., she may compute the expected utility of action a_i via

$$\psi(a_i) = \sum_{b_j \in \mathcal{B}} Q(a_i, b_j) p_A(b_j)$$

where $p_A(b_j)$ reflects A 's beliefs about her opponent's actions and is computed using the empirical frequencies of the opponent past plays. Then she may choose the action $a_i \in \mathcal{A}$ that maximizes her expected utility. In the following sections, we refer to this variant as FPQ-learning.

As described in (?), it is possible to re-frame fictitious play from a Bayesian perspective. Let p_j be the probability that the opponent chooses action b_j . We may place a Dirichlet prior $(p_1, \dots, p_n) \sim \mathcal{D}(\alpha_1, \dots, \alpha_n)$. Then, the posterior has the analytical form $\mathcal{D}(\alpha_1 + h_1, \dots, \alpha_n + h_n)$, with h_i being the count of action b_i , $i = 1, \dots, n$. If we denote the posterior density function as $f(p|h)$, then the DM would choose the action a_i maximizing her expected utility, which now takes the form

$$\begin{aligned} \psi(a_i) &= \int \left[\sum_{b_j \in \mathcal{B}} Q(a_i, b_j) p_j \right] f(p|h) dp \\ &= \sum_{b_j \in \mathcal{B}} Q(a_i, b_j) \mathbb{E}_{p|h} [p_j] \propto \sum_{b_j \in \mathcal{B}} Q(a_i, b_j) (\alpha_i + h_i). \end{aligned}$$

The Bayesian perspective may benefit the convergence of Q-learning, as we may include prior information about the adversary behavior when relevant.

Generalizing the previous approach to account for states is straightforward. Now the Q-function has the form $Q(a_i, b_j, s)$, where s is the state of the TMDP. The DM may need to assess probabilities of the form $p_A(b_j|s)$, since it is natural to expect that her opponent behaves differently depending on the state of the game and, consequently, depending also on previous actions. As before, the supported DM may choose her action at state s by maximizing

$$\psi_s(a_i) = \sum_{b_j \in \mathcal{B}} Q(a_i, b_j, s) p_A(b_j|s).$$

Since the state space $\mathcal{S}$ may be huge (or even continuous), keeping track of $p_A(b_j|s)$ may incur in prohibitive memory costs. Bayes rule may turn out to be useful, using

$$p_A(b_j|s) \propto p(s|b_j) p(b_j).$$

(?) propose an efficient method using a hash table or a bloom filter to maintain a count of the number of times an agent visits each state s , $p(s)$. This is only used in the context of single-agent RL to assist for better exploration of the environment. We propose to keep track of $|\mathcal{B}| = n$ bloom filters, one for each distribution $p(s|b_j)$, for tractable computation of the opponent's intentions in the TMDP setting.

The previous scheme may be transparently integrated with the Bayesian paradigm: we only need to store an additional array with the Dirichlet prior parameters α_i , $i = 1, \dots, n$ for the $p(b_j)$ part. Potentially, we could store initial pseudocounts as priors for each $b_j|s$ initializing the bloom filters with the corresponding parameter values.

If we assume the opponent to have memory of the previous stage actions, we could straightforwardly extend the previous scheme using the concept of mixtures of Markov chains, as described in (?). For example, in case the opponent belief model is $p_A(b_t|a_{t-1}, b_{t-1}, s_t)$, so that the adversary recalls the previous actions a_{t-1} and b_{t-1} , it could be factorized as a mixture

$$\begin{aligned} p_A(b_t|a_{t-1}, b_{t-1}, s_t) &= w_1 p_A(b_t|a_{t-1}) + \\ &\quad + w_2 p_A(b_t|b_{t-1}) + w_3 p_A(b_t|s_t). \end{aligned}$$

Then, if we allow for longer memories, instead of an exponential growth in the number of parameters, the complexity can be linearly controlled.

To conclude this section, we shall note that the described scheme is *model agnostic*, i.e., it does not matter if we represent the Q-function using a look-up table or a deep neural network (DQN), so we expect it to be usable in both shallow and deep multi-agent RL settings.

2.2 Level- k thinking

The previous section described how to model a level-0 opponent, i.e. a non strategic opponent, which can be practical in several scenarios. However, if the opponent is strategic, he may model the supported DM as a level-0 thinker, thus making the adversary a level-1 thinker. This chain can go up to infinity, so we will have to deal with modeling the opponent as a level- k thinker, with k bounded by the computational or cognitive resources of the DM.

To deal with it, we introduce a hierarchy of TMDPs in which $TMDP_i^k$ refers to the TMDP that agent i needs to optimize, while considering its rival as a level- $(k-1)$ thinker. Thus, we have the following process:

- If the supported DM is a level-1 thinker, she may optimize for $TMDP_A^1$. She then models B as a level-0 thinker (using Section 2.1).
- If the supported DM is a level-2 thinker, she may optimize for $TMDP_A^2$. She models B as a level-1 thinker. Consequently, this “modeled” B optimizes $TMDP_B^1$, and while doing so, he models the DM as level-0 (Section 2.1).
- In general, we have the chain of TMDPs:

$$TMDP_A^k \rightarrow TMDP_B^{k-1} \rightarrow \dots \rightarrow TMDP_B^1.$$

Exploiting the fact that we are in a repeated interaction setting (and by assumption that both agents can observe all past committed decisions and obtained rewards), each agent may estimate their counterpart’s Q-function, $\hat{Q}^{k-1}$: if the DM is optimizing $TMDP_A^k$, she will keep her own Q-function (we refer to it as Q_k), and also an estimate $\hat{Q}_{k-1}$, of her opponent’s Q-function. This estimate may be computed by optimizing $TMDP_B^{k-1}$ and so on until $k = 1$. Finally, the top level DM’s policy is given by

$$\arg \max_{a_{i_k}} Q_k(a_{i_k}, b_{j_{k-1}}, s),$$

where $b_{j_{k-1}}$ is now given by

$$\arg \max_{b_{j_{k-1}}} \hat{Q}_{k-1}(a_{i_{k-2}}, b_{j_{k-1}}, s)$$

and so on, until we arrive at the induction basis (level-1) in which the opponent may be modeled using the fictitious play approach from Section 2.1.

Note that in the previous hierarchy of policies the decisions are obtained in a greedy, deterministic manner (i.e. just by maximizing the lower level $\hat{Q}$ estimate). We may gain insight from the Bayesian / Risk Analysis communities by adding uncertainty to the policy at each level. For instance, at a certain level in the hierarchy, we could consider ϵ -greedy policies that with probability $1 - \epsilon$ choose an action according to the previous scheme, and with probability ϵ select a random action. Thus, we may impose distributions $p_k(\epsilon)$ at each level k of the hierarchy. The mean of $p_k(\epsilon)$ may be an increasing function with respect to the level k to account for the fact that in upper levels of thinking the uncertainty is higher. Other approaches to add uncertainty to the policies are left for future work.

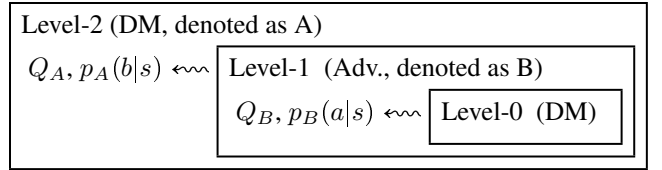


Figure 1: Level- k thinking scheme, with $k = 2$

Algorithm 1 specifies the approach for a level-2 DM. Because she is a level-2 DM, we need to account for her Q-function, Q_A (equivalently Q_2 from before), and that of her opponent (who will be level-1), Q_B (equivalently $\hat{Q}_1$). Figure 1 provides a schematic view of the dependencies.

3 Experiments and Results

To illustrate the TMDP’s and level- k reasoning framework, we consider two sets of experiments: repeated matrix games, with and without memory, and the adversarial environment proposed in (?). All the code is released at <https://github.com/vicgalle/ARAMARL>. The interested reader might check the previous repository or the Supplementary Material ?? for experimental setup details.

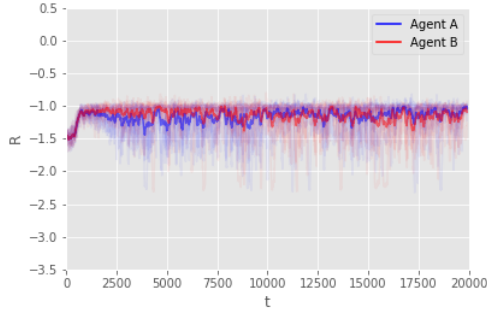
3.1 Repeated matrix games

Memoryless Repeated Matrix Games As an initial baseline, we focus on the stateless version of a TMDP. We consider the classical Iterated Prisoner’s Dilemma (IPD) (?), and analyze the policies learned by the supported DM, who will be the row player, against several kinds of opponents. Table 1 shows the reward bimatrix ($\mathbf{r}^A, \mathbf{r}^B$) for the Prisoner’s Dilemma. To construct the iterated game, we set the discount factor $\gamma = 0.96$ in the experiments so agent $i \in \{A, B\}$ aims at optimizing $\sum_{t=0}^{\infty} \gamma^t r_t^i$.

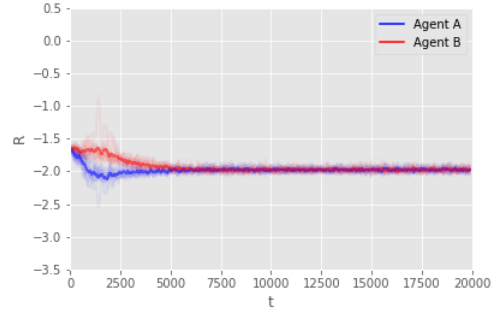
Table 1: Payoff Matrix of Prisoners’ Dilemma

	C	D
C	(-1, -1)	(-3, 0)
D	(0, -3)	(-2, -2)

To start with, we consider that the opponent is an independent-Q learner (i.e., he uses the standard Q-function from single-agent RL and Eq. 1 as learning rule). Figures 2(a) and 2(b) depict the utilities obtained over time, in cases where we model the DM as another independent Q-learner (Fig. 2(a)) or as a joint Q-learner with fictitious play (FPQ-learner), Fig. 2(b). Note that the FP playing solution (level-1) converges to the Nash equilibrium. The DM reaches the equilibrium strategy first, becoming stationary to her opponent, and thus pulling him to play towards the equilibrium strategy. In contrast, the opponent-unaware solution would remain exploitable by another adversary (i.e., independent Q-learning does not converge). Also note that in Fig. 2(a) the variance is much bigger due to the inability of the basic Q-learning solution to deal with a non-stationary environment.

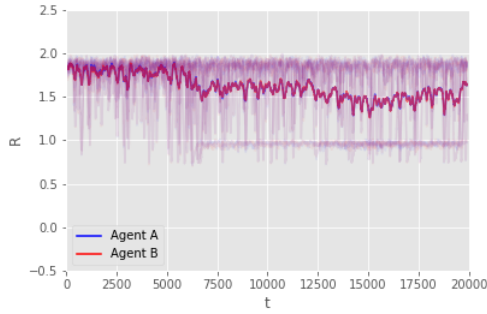


(a) Q-learner vs Q-learner



(b) FPQ-learner (blue) vs Q-learner (red)

Figure 2: Rewards obtained in IPD. We plot the trajectories of 10 simulations with shaded colors. Darker curves depict mean rewards along the 10 simulations.



(a) Q-learner vs Q-learner

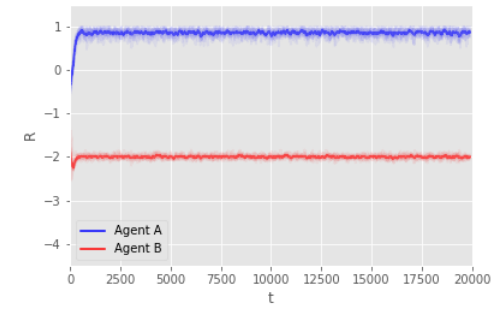


(b) FPQ-learner (blue) vs Q-learner (red)

Figure 3: Rewards in the iterated stag hunt game



(a) Q-learner vs Q-learner



(b) FPQ-learner (blue) vs Q-learner (red)

Figure 4: Rewards in the iterated chicken game

Algorithm 1 Level-2 thinking update rule

Require: $Q_A, Q_B, \alpha_A, \alpha_B$ (DM and opponent Q-functions and learning rates, respectively).

Observe transition (s, a, b, r_A, r_B, s') from the TMDP environment

$$Q_B(s, b, a) := (1 - \alpha_B)Q_B(s, b, a) + \alpha_B(r_B + \gamma \max_{b'} \mathbb{E}_{p_B(a'|s')} [Q_B(s', b', a')]) \quad \triangleright \text{Level-1}$$

Compute B's estimated ϵ -greedy policy $p_A(b|s')$ from $Q_B(s, b, a)$

$$Q_A(s, a, b) := (1 - \alpha_A)Q_A(s, a, b) + \alpha_A(r_A + \gamma \max_{a'} \mathbb{E}_{p_A(b'|s')} [Q_A(s', a', b')]) \quad \triangleright \text{Level-2}$$

We turn to another social dilemma game in which both agents must coordinate to maximize their rewards, the Stag Hunt game, with payoff matrix shown in Table 2. We focus on its iterated version referred to as ISH.

Table 2: Payoff Matrix of Stag Hunt

	C	D
C	(2, 2)	(0, 1)
D	(1, 0)	(1, 1)

We repeated the same experimental setting as in the IPD and report the results in Figure 3. Once again, the independent learning solution cannot tackle the non-stationarity of the environment, so it oscillates between the two Nash equilibria (C,C) and (D,D) without a clear convergence to one of them (Fig. 3(a)). On the other hand, the FPQ-learner converges earlier to the socially optimal policy. Then, the environment becomes essentially stationary for its opponent, who also converges to that policy.

The last social dilemma that we consider is the Chicken game, with payoff matrix in Table 3. This game has two pure Nash equilibria (C, D) and (D,C).

Table 3: Payoff Matrix of Chicken

	C	D
C	(0, 0)	(-2, 1)
D	(1, -2)	(-4, -4)

Results are reported in Figure 4. Figure 4(a) depicts again the ill convergence due to lack of opponent awareness in the independent Q-learning method. We noted that the instabilities continued cycling even after the limit in the displayed graphics. On the other hand, the DM with opponent modeling has an advantage and converges to her optimal Nash equilibrium (D,C) (Fig. 4(b)).

In addition, we also consider another kind of opponent to show our framework can adapt to it. We consider an adversary that learns according to the WoLF-PHC algorithm (?). Figure 5(a) depicts a FP-Q learner (level-1) against this adversary, where the latter exploits the former. However, if we go up in the level- k hierarchy and model the DM as a level-2 Q-learner, she outperforms her opponent (Fig. 5(b)).

Repeated Matrix Games With Memory In this section we give both players memory of past actions, in order to account for TMDP's with different states. We can augment the agents to have memory of the past K joint actions taken. However,

(?) proved that agents with a good memory-1 strategy can effectively force the iterated game to be played as memory-1, ignoring larger play histories. Thus, we resort to memory-1 iterated games here.

We may model the memory-1 IPD as a TMDP, in which the state S consists of elements of the form

$$s_t = (a_{t-1}, b_{t-1}), \quad t > 0$$

describing the previous joint action, plus the initial state s_0 in which there is no prior action. Note that now, the DM's policy is conditioned on S , so it may be fully specified by the $|S|$ probabilities $\pi(C|CC), \pi(C|CD), \pi(C|DC), \pi(C|DD), \pi(C|s_0)$.

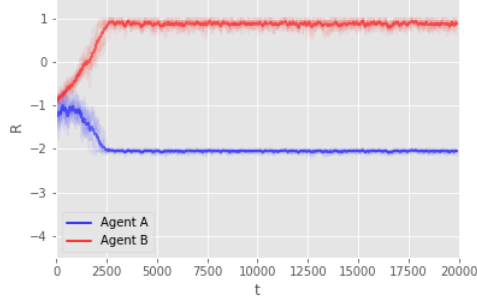
We assume an stationary adversary playing TitForTat (TFT), i.e. replicating the opponent's previous action, (?). He will compete with either another agent playing FP, or with a memory-1 agent also playing FP. In Figure 6 we represent the utilities perceived by these agents in both duels. As can be seen, a memoryless FPQ player cannot learn an optimal policy, and forces the TFT agent to play defect. In contrast, augmenting this agent to have memory of the previous move allows him to learn the optimal policy (TFT), that is, he learns to cooperate.

3.2 AI Safety Gridworlds

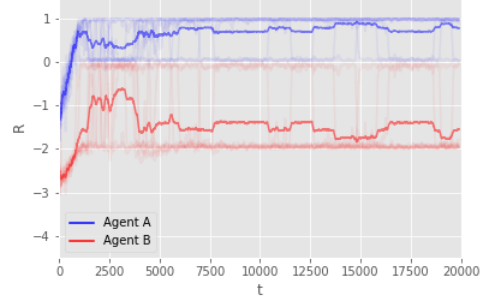
A suite of RL safety benchmarks was recently introduced in (?). We focus on the *friend or foe* environment, in which the supported DM needs to travel a room and choose between two identical boxes, hiding positive and negative rewards, respectively. This reward assignment is controlled by an adaptive adversary. Figure ?? shows the initial state in this game. The blue cell depicts the DM's initial state, gray cells represent the walls of the room. Cells 1 and 2 depict the adversary's targets, who will decide which one will hide the positive reward. This game may also be interpreted as a spatial Stackelberg game, in which the adversary is planning to attack one of two targets, and the defender (DM) will obtain a positive reward if she travels to the chosen target. Otherwise, she will miss the attacker and will incur in a loss.

As shown in (?), the *deep Q-network* (and similarly the independent tabular Q-learner as we will show) fails to achieve optimal results because the reward process is controlled by the adversary. We show that by explicitly modeling the adversary we actually improve Q-learning methods to achieve optimal utilities.

Stateless Variant We first consider a simplified environment with a singleton state and two actions. In a similar spirit to (?), the adaptive opponent estimates the DM's actions using an exponential smoother. Let $\mathbf{p} = (p_1, p_2)$ be the



(a) FPQ-learner (blue) vs WoLF-learner (red)



(b) L2Q-learner (blue) vs WoLF-learner (red)

Figure 5: Rewards obtained in the iterated chicken game against a WoLF-PHC adversary.

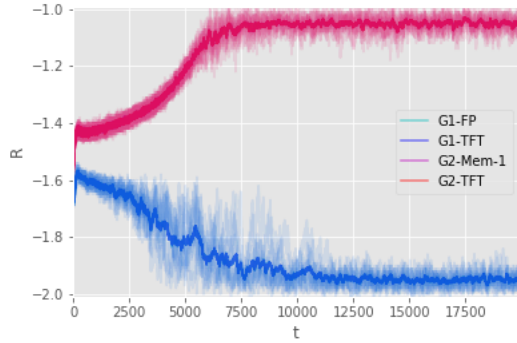


Figure 6: Rewards obtained for two different iterated games: TFT player vs FPQ memoryless player (G1) and TFT player vs FPQ memory-1 player (G2)

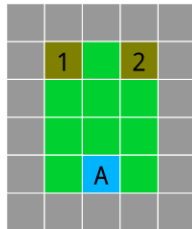


Figure 7: The *friend or foe* environment from the AI Safety Gridworlds benchmark. Figure taken from (?).

probabilities with which the DM will choose targets 1 or 2, respectively, as estimated by the opponent. Then, at every iteration he updates his knowledge through

$$\mathbf{p} := \alpha \mathbf{p} + (1 - \alpha) \mathbf{a}$$

where $0 < \alpha < 1$ is a learning rate, unknown from the DM's point of view, and $\mathbf{a} \in \{(1, 0), (0, 1)\}$ is a one-hot encoded vector indicating whether the DM has chosen targets 1 or 2. We consider an adversarial opponent which places the positive reward in target $t = \arg \min_i (\mathbf{p})_i$. As an example, in the beginning of a game, the opponent has estimate $\mathbf{p} = (0.5, 0.5)$ of the preferred target for the DM. If she chooses target 1, then opponent's estimate of p_1 will increase. Henceforth, in the next round he will place the positive reward in target 2.

Since the DM has to deal with an adaptive adversary, we introduce a modification to the FP-Q learning algorithm. Leveraging the property that the Dirichlet distribution is a conjugate prior of the Categorical distribution, a modified update scheme is proposed in Algorithm 2.

Algorithm 2 Dirichlet updating with a forget factor

Initialize pseudocounts $\alpha^0 = (\alpha_1^0, \dots, \alpha_K^0)$
for $t = 1, \dots, T$ **do**
 $\alpha^t = \lambda \alpha^{t-1}$ $\triangleright$ Reweight with factor $0 < \lambda < 1$
 Observe opponent action $b_i^t, i \in \{b_1, \dots, b_K\}$
 $\alpha_i^t = \alpha_i^{t-1} + 1$ $\triangleright$ Update posterior
 $\alpha_{-i}^t = \alpha_{-i}^{t-1}$
end for

It essentially allows to account for the last $\frac{1}{1-\lambda}$ opponent actions, instead of weighting all observations equally. For the case of a level-2 defender, as we do not know the actual rewards of the adversary (who will be modeled as a level-1 learner), we may model it as in a zero-sum scenario, i.e. $r_B = -r_A$. Other reward scalings for r_B were also considered, though they did not qualitatively affect the results (See Supplementary Material ??).

Results are displayed in Figure 8. We considered four types of defenders: opponent-agnostic Q-learner, a level-1

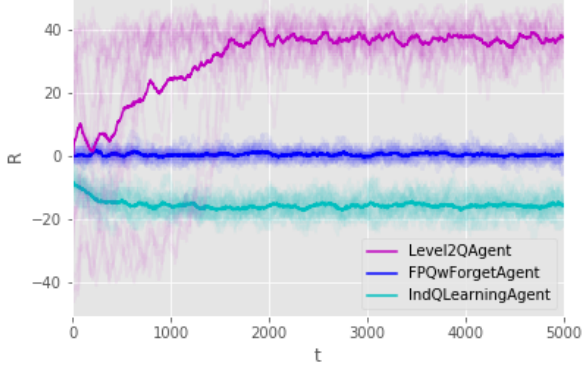


Figure 8: Rewards against the adversarial opponent

DM with forget and a level-2 agent. The first one is exploited by the adversary and, therefore, achieves suboptimal results. In contrast, the level-1 DM with forget effectively learns an stationary optimal policy (reward 0). Finally, the level-2 agent learns to exploit the adaptive agent achieving positive reward.

Note that the actual adversary behaves differently from how the DM models him, i.e. he is not exactly a level-1 Q-learner. Even so, modeling him as a level-1 agent gives the DM sufficient advantage.

Spatial Variant We now compare the independent Q-learner and a level-2 Q-learner against the same adaptive opponent in a spatial gridworld domain, Figure ?? . Targets' rewards are delayed until the DM arrives at one of the respective locations, obtaining ± 50 depending on the target chosen by the adversary. Each step is penalized with a reward of -1 for the DM. Results are displayed in Figure 9. Once again, the independent Q-learner is exploited by the adversary, getting even more negative rewards than in Figure 8 due to the penalty taken at each step. In contrast, the level-2 agent is able to approximately estimate the adversarial behavior, modeling him as a level-1 agent, thus being able to obtain positive rewards.

4 Conclusions and further work

We have introduced TMDPs, a novel variant of MDPs. This is an original framework to support decision makers who confront adversaries that interfere with the reward generating process in reinforcement learning settings. TMDP's aim to provide one-sided prescriptive support to a DM, maximizing her subjective expected utility, taking into account potential negative actions taken by an adversary. Some theoretical results are provided, in particular, we proved that our proposed learning rule is a contraction mapping so that we may use standard RL results of convergence. In addition, we propose a scheme to model adversarial behavior based on level- k reasoning about opponents. Further empirical evidence is provided via extensive experiments, with encouraging results.

Several lines of work are possible for further research.

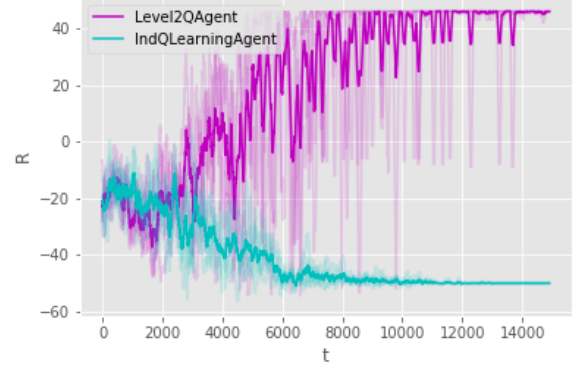


Figure 9: Rewards against the adversarial opponent in the spatial environment

First of all, we have limited to the case of facing just one adversary. The framework could be extended to the case of having multiple adversaries. In the experiments, we have just considered up to level-2 DM's, though the extension to higher order adversaries seems straightforward.

In addition, in recent years Q-learning has benefited from advances from the deep learning community, with breakthroughs such as the *deep Q-network* (DQN) which achieved super-human performance in control tasks such as Atari games (?), or as inner blocks inside systems that play Go (?). Integrating these advances into the TMDP setting is another possible research path. In particular, the proposed Algorithm 1 can be generalized to account for the use of deep Q-networks instead of tabular Q-learning.

Finally, it might be interesting to explore similar expansions of semi-MDPs, in order to perform Hierarchical RL or allow for time-dependent rewards and transitions between states.

5 Acknowledgments

R.N. acknowledges support from the Spanish Ministry for his grant FPU15-03636, V.G. acknowledges support from grant FPU16-05034. DRI is grateful to the MINECO MTM2014-56949-C3-1-R project and the AXA-ICMAT Chair in Adversarial Risk Analysis. All authors acknowledge support from the Severo Ochoa Excellence Programme SEV-2015-0554.

A provident nihil facilis aperiam eum laborum nostrum optio iste eius, labore illo doloremque id maxime at quas eligendi facilis neque alias eos, repellendus accusamus excepturi eius eveniet temporibus sunt, sit similique ad ut mollitia nobis eos soluta dicta vero deserunt? Odit non officia nemo expedita placeat enim nobis temporibus, excepturi enim quasi repellat nostrum molestiae quos beatae veniam ab. Dolore accusamus unde a suscipit quos consequatur maiores esse similique numquam, eius consectetur doloribus cumque facilis sint facere eveniet aut, facilis repellendus saepe neque ad corrupti nobis nesciunt tempore aspernatur ipsa optio, alias quasi nam voluptatem modi voluptatibus possimus corporis nisi. Magni adipisci incidunt officia tempore eaque illum quae

facilis dolor, ducimus vel assumenda cumque corrupti voluptatem, quasi id laudantium molestiae ad inventore autem ex explicabo ducimus?